Architecture decisions — AI Research Paper Assistant

Full design doc: https://claude.ai/artifact/V25T7EaNv86epMc6PTQZoS (as of 2026-10-31)
Local-to-production blueprint and interview talk-tracks: https://claude.ai/artifact/Vw24sTHniV1Qpr6UWwXLMa (as of 2026-11-01)

Area	Decision	Rationale
Inference	Ollama (default) or any OpenAI-compatible server (LM Studio, mlx_lm.server) behind one interface	Runtime swap via PA_RUNTIME; JSON-schema constrained decoding
Model	7–9B instruct with native tool calling, Q4_K_M; num_ctx set explicitly	Best quality per GB; small models degrade near window top
Orchestration	Native async state machine, Pydantic RunState, checkpoint per node; no LangChain	Visible prompts, testable nodes, resumable runs
Router	Single constrained-enum call (chat / search / deep_read / compare)	Planning becomes classification, which SLMs do well
Retrieval	Proxy-Pointer: embed breadcrumbed pointers, load whole sections; page numbers carried per section	Precise hits; SLM sees coherent sections; UI opens the cited page
Storage	One SQLite file: tables + FTS5 BM25 (stopword-filtered) + in-memory vector matrix (RRF fusion)	State, memory, vectors stay transactionally consistent
Memory	Working / short-term (incremental rolling summary) / episodic / semantic facts with provenance + invalidation	Context engineering under hard per-slot token budgets
Error recovery	Schema retry with validation error, loop detection, step cap, verify node, deterministic fallback	Every request returns a grounded answer
API / UI	Starlette + SSE typed events (runs survive tab close, replay by sequence); React 19 + Tailwind 4 built with esbuild	Fully testable offline; prebuilt UI needs no Node to run

Production direction (blueprint, 2026-11-01)

- Seams: five ports (InferencePort, VectorIndex, RunStore, JobQueue, EventBus); the agent loop, schemas and chunking stay unchanged.
- Pivot order: PostgreSQL + API/worker split → queues (RabbitMQ/SQS) and worker pools → vLLM (small/large tiers, prefix caching) → dedicated vector DB (Qdrant/Milvus) only when pgvector misses targets, with a transactional outbox.
- Observability: OpenTelemetry GenAI spans to Langfuse; nightly evals. Main threat: indirect prompt injection from uploaded PDFs, contained by read-only tenant-scoped tools and validated, citation-checked output.

Local hardware prototype (2026-11-01)

Target: MacBook Pro M4 Pro, 24 GB unified memory, 273 GB/s bandwidth, macOS 26. Model memory budget ~16 GB (GPU working-set share of 24 GB).

Profile	Model (Q4_K_M)	Context	Est. memory
balanced (default)	qwen2.5:7b-instruct / qwen3:8b / llama3.1:8b, auto-detected from pulled models	16k	~6 GB
quality	qwen2.5:14b-instruct	8k	~10.5 GB
fast	qwen2.5:3b-instruct	16k	~3 GB

- Embeddings: nomic-embed-text. Ollama tuning: flash attention, q8_0 KV cache, NUM_PARALLEL=1, MAX_LOADED_MODELS=2.
- Ingestion: one paper at a time via a persistent queue (papers table), so ingestion never competes with chat for the GPU.
- Parser: PyMuPDF4LLM by default (AGPL), Docling optional for tables.
- Location: ~/Agentic-AI/ai-paper-assistant. Setup: `bash scripts/setup_mac.sh`; run: `uv run python -m app serve`.
- Verified in the cloud workspace: 31 tests (unit, HTTP end-to-end, Ollama wire protocol) and a browser run against a mock Ollama. Not yet run against real models on the Mac.